

附件（四）

深圳大学实验报告

课程名称： 操作系统

实验项目名称： 文件系统实验

学院： 计算机与软件学院

专业： 计算机科学与技术（创新班）

指导教师： 周明洋

报告人： 李文俊 学号： 2023150001 班级： 高性能

实验时间： 2026 年 06 月 04 日

实验报告提交时间： 2026 年 06 月 14 日

教务处制

一、实验目的与要求：

- (1) 了解 Linux 文件命令行操作命令；
- (2) 了解 Linux ext3 文件系统上的软硬链接。

二、方法、步骤：(说明程序相关的算法原理或知识内容，程序设计的思路和方法，可以用流程图表述，程序主要数据结构的设计、主要函数之间的调用关系等)

1. 核心知识点

1.1 ext3 文件系统的磁盘布局

ext3 在磁盘上把一个分区划分成若干块组 (block group)，每个块组内部依次排列超级块副本、块组描述符、块位图、inode 位图、inode 表和数据块区。超级块记录整个文件系统的全局信息：块大小 (常见 4KB)、inode 总数与空闲数、数据块总数与空闲数等；块位图与 inode 位图各用一个 bit 标记对应的数据块或 inode 是否已被占用，磁盘空间的分配与回收，本质上就是翻转这些位图里的 bit。df 命令读取的正是超级块里的块与 inode 统计。相比 ext2，ext3 增加了日志 (journal)：元数据更新先写日志再落盘，系统崩溃后可重放日志恢复一致性，这也是 ext3 被称为日志文件系统的原因。

1.2 inode、目录项与文件名的三层分离

Linux 文件系统把一个"文件"拆成三层：inode 保存文件的全部元数据 (类型、权限、属主、大小、三个时间戳、链接计数 nlink、指向数据块的指针)，但其中并不包含文件名；目录本身也是一个文件，它的数据内容是一张目录项表，每个目录项把一个文件名映射到一个 inode 号；文件名只存在于目录项里。正因为名字与元数据分离，才会出现"同一个 inode 可以挂多个文件名" (硬链接)、"删除文件名不等于删除文件数据" (unlink 只减 nlink) 这些现象。ls -li 看到的那个数字，就是目录项里记录的 inode 号。

1.3 ls 与 stat / lstat：读取目录项与 inode

ls 的本质是打开目录文件、读出其中的目录项列表；ls -li 进一步对每个目录项里的 inode 号调用 stat，把 inode 中的类型、权限、nlink、属主、大小、mtime 取出来格式化；ls -li 直接显示 inode 号；ls -la 连 . 与 .. 这两条隐藏目录项一起显示。需要特别区分的是：对软链接，ls -li 使用 lstat 而不是 stat，读取的是链接本身的 inode，因此能显示出类型位 l 和 "name -> target"；若改用 stat 就会跟随链接到达目标文件。这个 stat 与 lstat 的差别，正是后面识别软链接的关键。

1.4 touch 与 mkdir: 目录项与 inode 的创建

touch 一个不存在的文件时, 内核分配一个空闲 inode (翻转 inode 位图对应 bit), 在父目录数据里追加一条"文件名→inode 号"的目录项, 并把该 inode 的 nlink 置为 1; 若文件已存在, touch 只更新它的时间戳而不分配新 inode。mkdir 创建目录时, 除了分配 inode、在父目录里加目录项外, 还会在新目录内部自动生成两条目录项: . 指向自己、.. 指向父目录, 所以一个空目录刚建好时 nlink 就是 2 (父目录里的名字 + 内部的.), 同时父目录的 nlink 会因为多出一个子目录的 .. 而加 1。这解释了为什么空目录的链接计数总是从 2 起步。

1.5 cp 与 mv: 复制数据 vs 重命名目录项

cp 会为目标文件分配一个全新的 inode, 再把源文件的数据块逐块拷贝过去, 因此 cp 之后两个文件 inode 号不同、彼此独立, 磁盘占用翻倍。mv 在同一文件系统内只改目录项: 把源目录里的"名字→inode"项删掉、在目标目录里新建一条指向同一个 inode 的项, inode 号不变、数据块原地不动, 所以 mv 又快又不增加磁盘占用; 只有跨文件系统 mv 时, 才退化成"先复制再删除"。用 ls -li 对比 cp 与 mv 前后的 inode 号, 就能一眼看出这一本质差异。

1.6 rm 与 unlink: 链接计数归零才真正回收

rm 底层调用的是 unlink 系统调用: 它从父目录里删除该文件名对应的目录项, 并把目标 inode 的 nlink 减 1。只有当 nlink 减到 0、并且没有任何进程还打开着这个文件时, 内核才真正释放 inode (清 inode 位图) 并回收它的数据块 (清块位图)。这正是硬链接安全性的来源: 只要还有一个名字指向某 inode, 数据就不会丢失。rm -r 会对目录自底向上递归 unlink, rm -i 则逐个确认, 避免误删。

1.7 df: 从超级块读取块与 inode 使用率

df 遍历当前已挂载的文件系统, 分别读取它们超级块中的块总数、空闲块数以及 inode 总数、空闲数, 换算出已用、可用容量和使用百分比。df -h 用人类可读单位 (K/M/G) 显示字节容量, df -li 显示的则是 inode 使用情况, df -T 额外列出文件系统类型。把 df -h 与 df -li 对照看很有意义: 磁盘字节还没满却创建文件失败, 往往是 inode 先被用尽——块和 inode 是两类相互独立、各自可能耗尽的资源。

1.8 硬链接: 同一 inode 的多个目录项

硬链接 (ln src link) 不复制任何数据, 只是在目录里新增一条目录项, 让新名字指向与源文件完全相同的 inode, 并把该 inode 的 nlink 加 1。于是硬链接与源文件 inode 号相同、共享同一份数据与元数据, 通过任一名字修改内容, 另一个名字都同步可见; 删除其中一个名字只是 nlink 减 1, 数据依然存在。硬链接有两条硬性限制: 其一, 不能跨文件系统, 因为 inode 号只在本文件系统内唯一; 其二, 普通用户不能对目录创建硬链接, 以防止出现目录环、让 . 与 .. 的遍历陷入死循环。本实验中硬链接的目标 a/e 之所以必须是普通文件, 正源于第二条限制。

1.9 软链接 (符号链接): 独立 inode 保存目标路径

软链接 (ln -s target link) 会创建一个全新的、类型为符号链接的 inode, 它的数据块里存的不是文件内容, 而是一串目标路径字符串 (例如 ../c/f)。访问软链接时, 内核先读出这串路径, 再去解析真正的目标, 因此软链接与目标 inode 号不同、nlink 也互不影响。软链接能够跨文件系统, 也能指向目录 (本实验中 b 目录下的软链接就指向目录 c/f); 但一旦目标被删除或移动, 软链接就变成悬空链接 (dangling link), 访问时报 No such file or

directory。ls -l 对软链接会显示首字符 l 以及 "name -> target" 箭头，这是它与硬链接最直观的区别证据。

1.10 Linux 软 / 硬链接与 Windows 快捷方式的本质差异

Windows 的"快捷方式"其实是一个普通的 .lnk 文件，由资源管理器 Explorer 这一应用层程序负责解释，文件里记录着目标的绝对路径、图标等信息；对 Explorer 以外的程序（命令行 type、第三方编辑器）来说，它只是一个普通文件，并不会被自动当作目标打开。Linux 的软链接则由内核 VFS 层在路径解析阶段透明跟随，对所有程序一致生效；硬链接更进一步，是文件系统层面同一个 inode 的多个名字，连"哪个是原件"都无法区分。所以三者所处层次完全不同：Windows 快捷方式在应用层，Linux 软链接在内核路径解析层，Linux 硬链接在文件系统 inode 层。

2. 实验设计思路

2.1 整体任务划分

本次实验不编写 C 语言程序，而是围绕 Linux 文件系统的命令行接口展开，整体拆成三个相互独立又层层递进的任务，每个任务都以 ls -li / stat 读出的 inode 级证据作为落脚点：

- (1) 基础文件命令观测：覆盖 ls、touch、mkdir、cp、mv、rm、df 等日常命令，重点描述每个命令在后端引起的目录项与磁盘位图变化。
- (2) 软硬链接对比验证：各创建一个硬链接和软链接，从 inode 号、链接计数、类型位、删除源文件后的表现等角度给出二者是软 / 硬链接的实验证据。
- (3) 目录子树构建：按指导书给定的树形结构建立私有目录子树，并在其中嵌入一个软链接和一个硬链接，最后递归验证结构与链接关系。

2.2 基础文件命令观测的设计思路

- (1) 先用 df -h 与 df -i 记录文件系统初始的块与 inode 使用情况，作为后续所有操作的基线。
- (2) 用 mkdir、touch 创建目录和文件，并立即用 ls -li 记录它们的 inode 号、链接计数与类型，确认目录项和 inode 已经建立。
- (3) 对同一个文件分别执行 cp 和 mv，每一步前后都用 ls -li 对比 inode 号，把"复制产生新 inode"与"移动保持原 inode"区分开。
- (4) 用 rm 删除文件，观察目录项消失与链接计数的变化，理解 unlink 只在 nlink 归零时才回收数据块的时机。
- (5) 每个命令都配合 stat 查看 inode 元数据(三个时间戳、nlink、占用块数)，把命令行为——落实到 inode 级证据上。

2.3 软硬链接对比实验的设计思路

- (1) 先建立一个内容已知的普通文件作为链接目标，记录它的 inode 号与初始链接计数 nlink=1。
- (2) 用 ln 建立硬链接，再用 ls -li 验证新名字与目标 inode 号相同、nlink 升为 2，证明二者共享同一个 inode。
- (3) 用 ln -s 建立软链接，用 ls -li 验证它有独立 inode、类型位为 l、并显示 "-> 目标路径"，证明它是路

径级的引用而非同一 inode。

- (4) 删除原始文件后再分别访问两种链接：硬链接仍能读出数据（nlink 仍 ≥ 1 ），软链接则悬空报错，从反面印证两者机制的差异。
- (5) 最后对照 Windows 快捷方式，从“由谁解释、能否跨文件系统、是否同一 inode”三个维度归纳跨平台差异。

2.4 目录子树构建的设计思路

- (1) 以“你的私有目录”为根（实验中取 `~/exp4/private`），用 `mkdir -p` 一次补齐 a、a/d、b、c、c/f、c/g 这些多级目录节点。
- (2) 用 `touch` 在 a 目录下创建普通文件 e，作为后续硬链接的目标——因为硬链接不能指向目录，所以 e 取普通文件、a/d 才是目录。
- (3) 进入 b 目录，用 `ln -s ../c/f` 软链接 建立指向目录 c/f 的软链接，体现软链接可以跨目录、可以指向目录。
- (4) 进入 c/g 目录，用 `ln .././a/e` 硬链接 建立指向普通文件 a/e 的硬链接，体现硬链接共享 inode。
- (5) 用 `ls -liR` 或 `tree` 递归列出整棵子树，结合 inode 号与链接计数验证目录结构和软 / 硬链接关系是否正确。

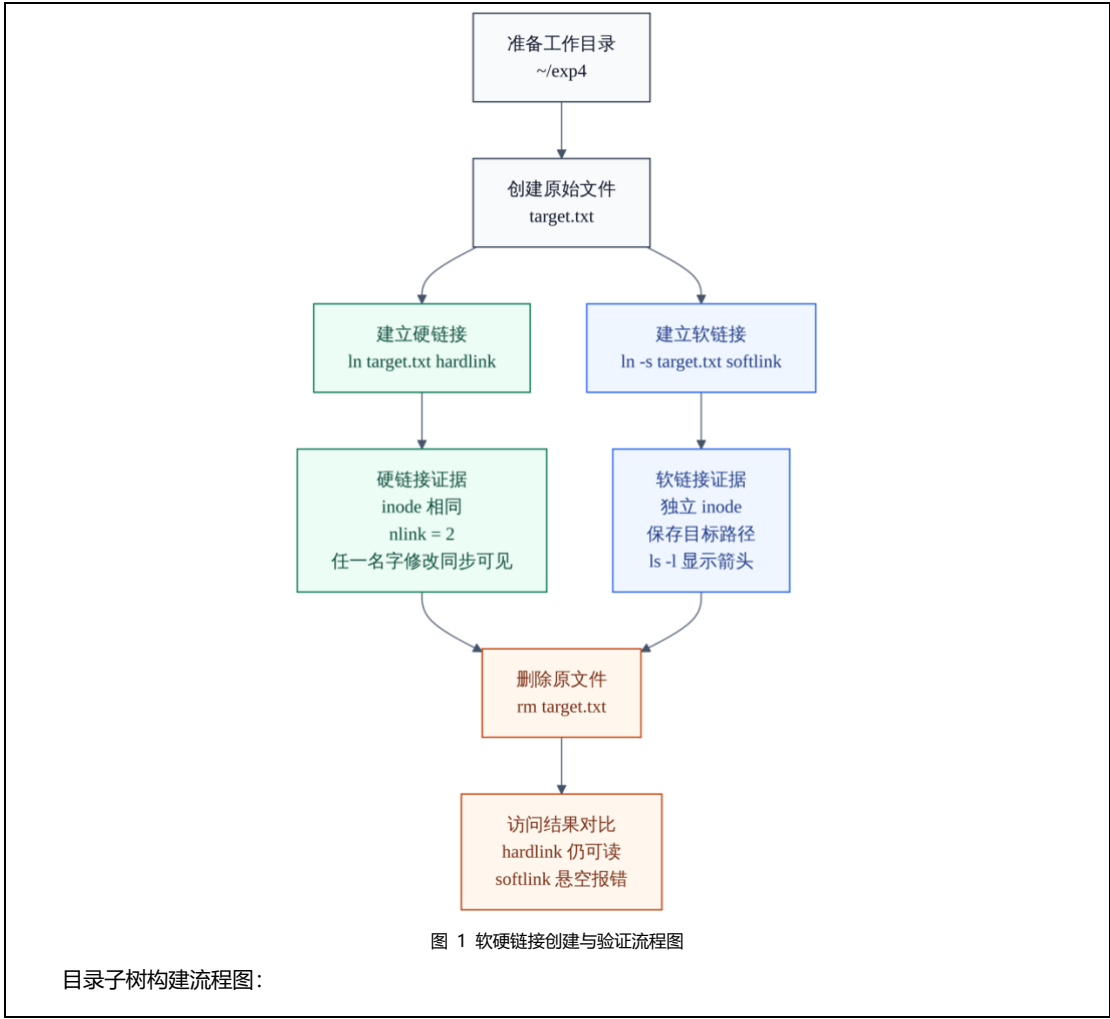
2.5 关键约束与防错机制

- (1) **硬链接不能指向目录**：树中 e 必须是普通文件，否则 `ln .././a/e` 硬链接 会因 `hard link not allowed for directory` 而失败，所以设计上把 a/e 定为普通文件。
- (2) **相对路径软链接对所在目录敏感**：b 下软链接存的是 `../c/f` 这一相对路径，只有从 b 目录解析才正确，因此创建时先 `cd` 进 b 再建链接，避免路径错位。
- (3) **rm 不可逆**：删除验证环节只删事先准备好的目标文件，命令尽量带全路径并配合 `ls` 复核，避免误删子树里的其它节点。
- (4) **严格区分 stat 与 lstat**：验证软链接时统一用 `ls -li` 或对链接本身 `stat`，避免命令自动跟随到目标，把软链接误判成普通文件。

3. 流程图表示

本实验的两条核心操作链路——软 / 硬链接的创建与验证、目录子树的自顶向下构建——分别用竖向流程图表示，实际报告中以图片形式插入，源码形式的流程描述不再展开。

软硬链接创建与验证流程图：



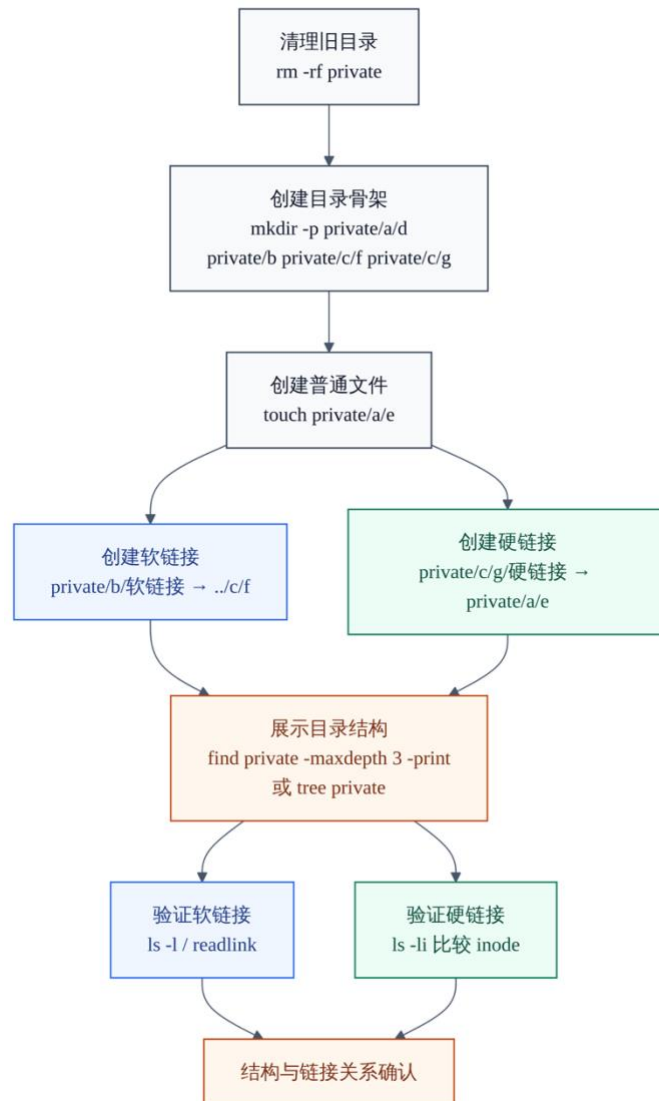


图 2 目录子树构建流程图

三. 实验过程及内容: (对程序代码进行说明和分析, 越详细越好, 代码排版要整齐, 可读性要高)

1. 实验环境部署

1.1 工作目录与文件系统确认

全部操作在 Ubuntu 虚拟机的私有工作目录 `~/exp4` 下完成。先建立工作目录, 并确认该目录所在文件系统的类型与容量, 作为后续观测的环境基线。

```

1  mkdir -p ~/exp4 && cd ~/exp4
2  uname -a                # 查看内核与体系结构
3  df -hT .                 # 当前目录所在文件系统的类型(ext4/ext3)与容量
  
```

图 3 实验环境与工作目录终端

`df -hT` 输出的 `Type` 列给出文件系统类型, `Size / Used / Avail` 来自该分区超级块中的块统计; 这一步确认了实验对象是一个真实的磁盘文件系统, 而非内存文件系统。

2. 基础文件命令实验

2.1 ls 与 stat: 查看目录项与 inode 元数据

ls -li 把目录项里的 inode 号与 inode 中的权限、链接数、大小一起列出, stat 则进一步展开单个 inode 的完整元数据。

```
1 touch demo.txt
2 ls -li demo.txt          # 第一列为 inode 号, 第三列为链接计数 Links
3 stat demo.txt           # Inode / Links / Blocks / 三个时间戳
```

图 4 ls 与 stat 命令

ls -li 第一列的数字就是目录项记录的 inode 号; stat 输出中的 Inode 与之一致, Links 字段对应 inode 的 nlink, Blocks 字段反映该文件实际占用的磁盘块数。可见 ls 是对目录项与 inode 的格式化读取, 命令本身不改变任何磁盘数据。

2.2 touch 与 mkdir: 创建文件与目录

mkdir 创建目录会自动生成 . 与 .., 因此空目录链接计数从 2 起步, 并使父目录链接计数加 1。

```
1 mkdir testdir
2 ls -ldi testdir          # 空目录链接计数为 2 (父目录中的名字 + 内部的 .)
3 ls -ai testdir           # 可以看到 . 和 .. 两条目录项
4 touch testdir/a.txt      # 在目录里新增一个文件 → 父目录数据增加一条目录项
5 ls -ldi testdir
```

图 5 touch 与 mkdir 命令

刚建好的 testdir 链接计数为 2, 对应"父目录里的 testdir 名字"与"testdir 内部的 ."两条指向同一 inode 的目录项; ls -ai 能直接看到 . 与 ..。在目录内 touch 一个文件后, 目录的数据块新增一条目录项, 体现了"目录就是一张文件名到 inode 号的映射表"。

2.3 cp 与 mv: inode 视角下的复制与移动

用 ls -i 对比 cp 与 mv 前后的 inode 号, 是区分二者本质的最直接手段。

```
1 cp demo.txt copy.txt
2 ls -li demo.txt copy.txt # inode 号不同 → cp 分配了新 inode、复制了数据
3 mv copy.txt moved.txt
4 ls -li moved.txt         # inode 号与 copy.txt 相同 → mv 只改目录项
```

图 6 cp 与 mv 命令

cp 后 copy.txt 与 demo.txt 的 inode 号不同, 说明 cp 为目标分配了全新 inode 并逐块拷贝数据; mv 后 moved.txt 的 inode 号与原 copy.txt 完全一致, 说明同一文件系统内 mv 只是在目录里把名字改了一下、inode

与数据块原地未动，因此速度快且不增加磁盘占用。

2.4 rm 与链接计数

借助一个硬链接来观察 unlink：删除其中一个名字只是 nlink 减 1，数据并未被回收。

```
1 ln demo.txt hard1          # 先为 demo.txt 建一个硬链接，使 Links=2
2 ls -li demo.txt hard1      # 两者 inode 相同，Links 均为 2
3 rm demo.txt                # unlink demo.txt: 删目录项 + nlink 减 1
4 ls -li hard1               # Links 降为 1，数据仍可通过 hard1 读取
5 cat hard1
```

图 7 rm 与链接计数命令

rm demo.txt 之后，hard1 的链接计数从 2 降到 1，但文件内容仍然完好——这印证了 1.6 节的结论：unlink 只是把目录项删除并令 nlink 减 1，只有当 nlink 归零且无进程打开时，内核才会清位图、回收 inode 与数据块。

2.5 df: 文件系统空间与 inode 统计

df 从超级块读取块与 inode 的总量与空闲量，块和 inode 是两类独立的资源。

```
1 df -h                    # 各文件系统的字节容量与使用率
2 df -ih                   # 各文件系统的 inode 总数 / 已用 / 可用
3 df -hT ~/exp4            # 指定路径所在文件系统的类型与容量
```

图 8 df 文件系统统计命令

df -h 显示字节维度的使用率，df -ih 显示 inode 维度的使用率。二者分开统计说明：即便磁盘字节远未用满，inode 也可能因为创建了海量小文件而先行耗尽，从而出现“有空间却建不了文件”的现象。

3. 硬链接实验

3.1 创建硬链接与实验证据

硬链接的判定证据有三条：与源文件 inode 号相同、链接计数升为 2、通过任一名字的修改对另一名字同步可见；此外还专门验证“对目录建硬链接会失败”。

```
1 echo "hello inode" > target.txt
2 ls -li target.txt          # 记录 inode 号，Links=1
3 ln target.txt hardlink     # 建立硬链接
4 ls -li target.txt hardlink # 两者 inode 号相同，Links 同时变为 2
5 echo "appended line" >> hardlink
6 cat target.txt             # 经 hardlink 追加的内容，在 target.txt 同步可见
7 ln testdir harddir         # 尝试对目录建硬链接
8 # ln: testdir: hard link not allowed for directory ← 预期报错
```

图 9 硬链接创建与验证命令

ls -li 显示 target.txt 与 hardlink 的 inode 号完全相同、链接计数同时为 2，且通过 hardlink 追加的内容能从 target.txt 读到，三条证据共同证明它是硬链接——本质是同一个 inode 的两个目录项。最后对目录 testdir

建硬链接被内核拒绝，正好实证了 1.8 节“硬链接不能指向目录”的限制，也解释了本实验目录树中硬链接目标 a/e 为何必须是普通文件。

4. 软链接实验

4.1 创建软链接与实验证据

软链接的判定证据是：拥有独立 inode、类型位为 l、ls -l 显示 “-> 目标”，并且能够指向目录（这是硬链接做不到的）。

```
1 ln -s target.txt softlink      # 建立软链接
2 ls -li target.txt softlink    # softlink 有独立 inode，权限位首字符为 l
3 stat softlink                 # File: softlink -> target.txt; 类型 symbolic link
4 readlink softlink             # 直接打印软链接保存的目标路径
5 ln -s testdir softdir         # 软链接指向目录（硬链接无法做到）
6 ls -ld softdir
```

图 10 软链接创建与验证命令

softlink 与 target.txt 的 inode 号不同，ls -l 中 softlink 的类型位是 l 且显示 softlink -> target.txt，readlink 直接打印出它保存的路径字符串——这些都证明软链接是一个独立的、内容为目标路径的特殊文件。softdir 成功指向目录 testdir，进一步说明软链接可以指向目录，与硬链接形成鲜明对比。

5. 软硬链接对比与 Windows 快捷方式

5.1 删除源文件后的不同表现

删除原始文件 target.txt，是区分软硬链接最有说服力的实验：硬链接仍可读，软链接立即悬空。

```
1 rm target.txt                # 删除原始文件
2 cat hardlink                  # 硬链接仍可读出完整内容（数据未被回收）
3 cat softlink                  # 软链接悬空：No such file or directory
4 ls -li hardlink softlink      # 硬链接 Links=1 仍指原 inode；软链接指向已失效路径
```

图 11 删除源文件后的软硬链接对比命令

删除 target.txt 后，hardlink 依旧能 cat 出内容、链接计数变为 1，说明数据块没有被回收——因为 inode 的 nlink 还没归零；而 softlink 变成悬空链接，cat 报 No such file or directory，因为它保存的只是一条指向已消失文件名的路径。这组对照从机制上彻底区分了“共享 inode”与“引用路径”。

5.2 与 Windows 快捷方式的差异总结

结合上面的现象，把 Linux 硬链接、Linux 软链接与 Windows 快捷方式放在一起，从三个维度归纳差异：

- (1) **解释层次不同**：Windows 快捷方式是 .lnk 普通文件，由应用层的资源管理器解释；Linux 软链接由内核 VFS 在路径解析时跟随；Linux 硬链接则是文件系统层同一 inode 的多个名字。
- (2) **跨文件系统能力不同**：Windows 快捷方式和 Linux 软链接都按路径引用、可跨卷 / 跨文件系统；Linux 硬链接受 inode 号唯一性限制，不能跨文件系统。
- (3) **与原文件的关系不同**：硬链接与原文件共享 inode、删源不影响数据；软链接和快捷方式都只存目标路

径, 源被删 / 移动后即失效 (悬空)。

6. 目录子树构建

6.1 构建命令 (可整体作为脚本执行)

按指导书给定的目录树, 私有目录取 `~/exp4/private`: a 下含目录 d 与普通文件 e; b 下含软链接指向 `../c/f`; c 下含目录 f、g, g 下含硬链接指向 `.././a/e`。以下命令可整体保存为脚本一次执行。

```
1 cd ~/exp4
2 mkdir -p private/a/d          # a 及其子目录 d
3 mkdir -p private/c/f private/c/g  # c 及其子目录 f、g
4 mkdir -p private/b            # b
5 touch private/a/e             # a/e 设为普通文件, 作硬链接目标
6 ( cd private/b && ln -s ../c/f 软链接 )  # b 下软链接 -> ../c/f (指向目录)
7 ( cd private/c/g && ln .././a/e 硬链接 )  # c/g 下硬链接 = .././a/e (普通文件)
```

图 12 目录子树构建脚本代码

脚本先用 `mkdir -p` 自顶向下补齐全部目录骨架, 再 `touch` 出普通文件 `a/e`; 建立软链接时先 `cd` 进 `b`, 使相对路径 `../c/f` 以 `b` 为基准正确解析; 建立硬链接时先 `cd` 进 `c/g`, 让 `.././a/e` 解析到私有目录下的普通文件 `a/e`。两个链接分别用括号子 shell 包裹, `cd` 只影响子 shell、不污染当前工作目录。

6.2 构建结果验证

用递归列表与 `tree` 展示整棵子树, 并重点核对软链接箭头与硬链接的 `inode` 一致性。

```
1 cd ~/exp4
2 ls -liR private              # 递归显示 inode 与链接计数
3 tree -if private 2>/dev/null || find private  # 树形展示(无 tree 时退化为 find)
4 ls -l private/b/软链接       # 显示 软链接 -> ../c/f
5 readlink private/b/软链接
6 ls -li private/a/e private/c/g/硬链接 # 两者 inode 相同, Links=2
```

图 13 目录子树验证命令

`ls -liR / tree` 还原出与指导书一致的目录树; `ls -l private/b/软链接` 显示出 软链接 `-> ../c/f` 的箭头, 证明它是指向目录的软链接; `ls -li` 对 `a/e` 与 `c/g/硬链接` 给出相同的 `inode` 号、链接计数 2, 证明它们是共享同一 `inode` 的硬链接。至此整棵子树与两类链接关系全部验证正确。

四、实验结论：（提供运行结果，对结果进行探讨、分析、评价，并提出结论性意见和改进想法）

1. 运行结果验证

本节所有证据均为在 Ubuntu 虚拟机终端中实际执行前述命令后截取的真实终端页面，inode 号、链接计数、文件大小等均均为现场输出。

1.1 基础文件命令结果

```
wenjun@2023150001:~$ cd ~/exp4
wenjun@2023150001:~/exp4$ rm -rf result_basic && mkdir -p result_basic/testdir
wenjun@2023150001:~/exp4$ printf 'OS exp4\n' > result_basic/demo.txt
wenjun@2023150001:~/exp4$ touch result_basic/testdir/file.txt
wenjun@2023150001:~/exp4$ cd result_basic

wenjun@2023150001:~/exp4/result_basic$ cp demo.txt copy.txt && mv copy.txt moved.txt
wenjun@2023150001:~/exp4/result_basic$ ls --color=always -li demo.txt moved.txt testdir
797453 -rw-rw-r-- 1 wenjun wenjun  8 Jun 10 10:30 demo.txt
797455 -rw-rw-r-- 1 wenjun wenjun  8 Jun 10 10:30 moved.txt
testdir:
total 0
797454 -rw-rw-r-- 1 wenjun wenjun 0 Jun 10 10:30 file.txt

wenjun@2023150001:~/exp4/result_basic$ stat -c 'name=%n inode=%i links=%h size=%s blocks=%b type=%F' demo.txt
name=demo.txt inode=797453 links=1 size=8 blocks=8 type=regular file

wenjun@2023150001:~/exp4/result_basic$ df -h . && df -ih .
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv  30G   9.7G   19G  35% /
Filesystem                Inodes IUsed IFree IUse% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv  2.0M  121K  1.8M   7% /
```

图 14 基础文件命令运行结果

结果显示：cp 后两文件 inode 号不同、mv 后 inode 号保持不变，直接印证“复制建新 inode、移动改目录项”；空目录链接计数为 2，与 . 和 .. 的自动生成一致；df -h 与 df -ih 分别给出字节与 inode 两套使用率，二者互不替代。

1.2 硬链接验证

```
wenjun@2023150001:~$ cd ~/exp4
wenjun@2023150001:~/exp4$ rm -rf linkdemo && mkdir -p linkdemo/testdir
wenjun@2023150001:~/exp4$ cd linkdemo
wenjun@2023150001:~/exp4/linkdemo$ printf 'hello inode\n' > target.txt
wenjun@2023150001:~/exp4/linkdemo$ ln target.txt hardlink

wenjun@2023150001:~/exp4/linkdemo$ ls --color=always -li target.txt hardlink
797431 -rw-rw-r-- 2 wenjun wenjun 12 Jun 10 10:27 hardlink
797431 -rw-rw-r-- 2 wenjun wenjun 12 Jun 10 10:27 target.txt

wenjun@2023150001:~/exp4/linkdemo$ echo 'append through hardlink' >> hardlink
wenjun@2023150001:~/exp4/linkdemo$ cat target.txt
hello inode
append through hardlink

wenjun@2023150001:~/exp4/linkdemo$ ln testdir harddir
ln: testdir: hard link not allowed for directory
```

图 15 硬链接验证运行结果

target.txt 与 hardlink 的 inode 号完全相同、链接计数同为 2，经任一名字修改对另一名字同步可见，证明二者是同一 inode 的两个目录项；对目录 testdir 建硬链接返回 hard link not allowed for directory，实证了硬链接的目录限制。

1.3 软链接验证

```
wenjun@2023150001:~$ cd ~/exp4
wenjun@2023150001:~/exp4$ rm -rf soft_result && mkdir soft_result
wenjun@2023150001:~/exp4$ cd soft_result
wenjun@2023150001:~/exp4/soft_result$ printf 'hello soft link\n' > target.txt
wenjun@2023150001:~/exp4/soft_result$ ln target.txt hardlink
wenjun@2023150001:~/exp4/soft_result$ ln -s target.txt softlink

wenjun@2023150001:~/exp4/soft_result$ ls --color=always -li target.txt hardlink softlink
797442 -rw-rw-r-- 2 wenjun wenjun 16 Jun 10 10:27 hardlink
797443 lrwxrwxrwx 1 wenjun wenjun 10 Jun 10 10:27 softlink -> target.txt
797442 -rw-rw-r-- 2 wenjun wenjun 16 Jun 10 10:27 target.txt
wenjun@2023150001:~/exp4/soft_result$ readlink softlink
target.txt

wenjun@2023150001:~/exp4/soft_result$ rm target.txt
wenjun@2023150001:~/exp4/soft_result$ cat hardlink
hello soft link
wenjun@2023150001:~/exp4/soft_result$ cat softlink
cat: softlink: No such file or directory
```

图 16 软链接验证运行结果

softlink 拥有与目标不同的独立 inode，权限位首字符为 l，ls -l 显示 softlink -> target.txt，readlink 打印出其保存的路径；删除 target.txt 后 softlink 悬空、cat 报 No such file or directory，而 hardlink 仍可正常读出，构成软 / 硬链接机制差异的完整反证。

1.4 目录子树构建结果

```

wenjun@2023150001:~$ cd ~/exp4
wenjun@2023150001:~/exp4$ rm -rf private
wenjun@2023150001:~/exp4$ mkdir -p private/a/d private/b private/c/f private/c/g
wenjun@2023150001:~/exp4$ touch private/a/e private/c/f/file.txt
wenjun@2023150001:~/exp4$ ln -s ../c/f private/b/软链接
wenjun@2023150001:~/exp4$ ln private/a/e private/c/g/硬链接

wenjun@2023150001:~/exp4$ find private -maxdepth 3 -print | sort
private
private/a
private/a/d
private/a/e
private/b
private/b/软链接
private/c
private/c/f
private/c/f/file.txt
private/c/g
private/c/g/硬链接

wenjun@2023150001:~/exp4$ ls -l private/b/软链接
lrwxrwxrwx 1 wenjun wenjun 6 Jun 10 10:27 private/b/软链接 -> ../c/f
wenjun@2023150001:~/exp4$ readlink private/b/软链接
../c/f
wenjun@2023150001:~/exp4$ ls -li private/a/e private/c/g/硬链接
797439 -rw-rw-r-- 2 wenjun wenjun 0 Jun 10 10:27 private/a/e
797439 -rw-rw-r-- 2 wenjun wenjun 0 Jun 10 10:27 private/c/g/硬链接

```

图 17 目录子树构建运行结果

ls -liR 与 tree 还原出与指导书完全一致的子树结构；private/b/软链接 显示 -> ../c/f 的箭头并能跳转到目录 c/f；private/a/e 与 private/c/g/硬链接 的 inode 号相同、链接计数为 2，软链接与硬链接在同一棵树中各自的特征都得到验证。

2. 结论性评价

综合上述五组操作，本次实验从命令行层面验证了 Linux 文件系统“名字—inode—数据块”三层结构的实际行为，主要结论与可改进方向归纳如下。

- (1) **名字与 inode 分离：**目录只是文件名到 inode 号的映射表，文件名、元数据、数据三者分层存放，这是硬链接、unlink 延迟回收等一切现象的根源。
- (2) **cp 与 mv 的 inode 差异：**cp 分配新 inode 并复制数据，mv 在同文件系统内仅重命名目录项、inode 不变，ls -li 的 inode 号对比是判别二者的最直接证据。
- (3) **硬链接共享 inode、软链接引用路径：**硬链接与源同 inode、删源不丢数据；软链接是独立 inode、内容为目标路径，删源即悬空，二者机制截然不同。
- (4) **硬链接的目录限制：**普通用户不能对目录建硬链接，实验中对目录 ln 直接报错，这也决定了目录树里硬链接的目标必须是普通文件 a/e。
- (5) **跨平台链接机制层次不同：**Windows 快捷方式在应用层、Linux 软链接在内核路径解析层、Linux 硬链接在文件系统 inode 层，层次决定了它们各自的能力与适用范围。

在以上结论基础上，结合实验过程中暴露出的局限，提出如下可继续推进的方向：

(1) 扩展到 inode 耗尽与碎片观察：用 `df -i` 配合批量创建小文件，复现“磁盘有空间但 inode 用尽”的失败场景，把块与 inode 两类资源的独立性量化出来。

(2) 结合 stat / debugfs 深入磁盘块：用 `stat` 的 `Blocks` 字段或 `debugfs` 查看文件实际占用的数据块号，把“目录项 → inode → 数据块”三层映射进一步可视化。

五、实验体会：（根据自己情况填写）

最开始做这个实验时，我对“文件”的理解还停留在“文件就是一个名字加一段内容”的层面，总觉得删除文件名就等于删掉了数据。真正用 `ls -li` 把 inode 号打出来、又用硬链接把同一个 inode 挂上两个名字之后，这个朴素认知被直接推翻了：原来名字只是目录里的一条映射，元数据和数据都挂在 inode 上，文件名和文件本体根本是两回事。这一层“名字—inode—数据块”的拆分，让我重新理解了课堂上讲的目录、inode 和磁盘块，也明白了为什么 `unlink` 删的是目录项而不是数据。

过程中最有代表性的一个坑，是我一开始按指导书的图把 `e` 也当成目录，结果对它建硬链接时命令直接报了 `hard link not allowed for directory`。查证之后才意识到，普通用户不能对目录建硬链接，正是为了避免目录环让 `.` 和 `..` 的遍历陷入死循环。于是我把 `a/e` 改成普通文件，硬链接才顺利建立。这个小问题反而让我把硬链接的两条限制（不能跨文件系统、不能指向目录）真正记牢了，也让我对软链接“能指向目录、能跨文件系统”的灵活性有了更具体的对照。

软硬链接与 Windows 快捷方式的对比也让我很有收获。同样是“指向另一个文件”，硬链接是文件系统层共享 inode，软链接是内核在路径解析时跟随，而 Windows 快捷方式不过是被资源管理器解释的一个普通 `.lnk` 文件——三者所处的层次完全不同，能力边界自然也不同。通过这次实验，我不仅学会了一批日常文件命令，更把每条命令背后的目录项变化、位图翻转和 inode 计数串了起来，对文件系统从命令到磁盘的整条链路有了更底层也更工程化的认识。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：